metaprogramming/accessor_2.rb

```
module AttrLogger
  def attr_logger(name)
    attr_reader name

    define_method("#{name}=") do |val|
      puts "Assigning #{val.inspect} to #{name}"
      instance_variable_set("@#{name}", val)
    end
  end
end

class Example
  extend AttrLogger
  attr_logger :value
end

ex = Example.new
ex.value = 123
puts "Value is #{ex.value}"
ex.value = "cat"
puts "Value is now #{ex.value}"
```

produces:

```
Assigning 123 to value
Value is 123
Assigning "cat" to value
Value is now cat
```

Things get a little trickier if you want to add both class methods and instance methods into the class being defined. Here's one technique that is used extensively in the implementation of Rails. It makes use of a Ruby hook method, included, which is called automatically by Ruby when you include a module into a class. It is passed the class object of the class being defined:

metaprogramming/accessor_3.rb

```
module GeneralLogger
  # Instance method to be added to any class that includes us
  def log(msg)
    puts Time.now.strftime("%H:%M: ") + msg
  end

  # module containing class methods to be added
  module ClassMethods
    def attr_logger(name)
      attr_reader name

      define_method("#{name}=") do |val|
        log "Assigning #{val.inspect} to #{name}"
        instance_variable_set("@#{name}", val)
      end
    end
  end

  # extend host class with class methods when we're included
  def self.included(host_class)
    host_class.extend(ClassMethods)
  end
end
```